练习

- 最大容量为n的循环队列，队尾指针是rear，队头指针是front，则队空的条件是_____，队满的条件是_____。
- 假定front和rear分别为一个链式队列的队头和队尾指针，则该链式队列中只有一个结点的条件为_____。
- 从一个栈顶指针为top的非空链式栈中删除结点并不需要返回栈顶结点的值和回收结点时，应执行_____操作（带表头结点）。
- 向一个栈顶指针为top的链式栈（无头结点）中插入一个新结点*p时，应执行_____和_____操作。

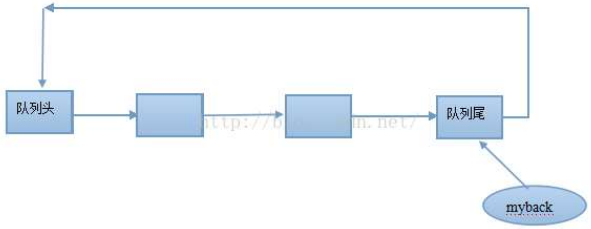
练习

- 最大容量为n的循环队列，队尾指针是rear，队头指针是front，则队空的条件是_____ rear == front _____，队满的条件是_____ (rear+1) % n==front ____。
- 假定front和rear分别为一个链式队列的队头和队尾指针，则该链式队列中只有一个结点的条件为____ front->next == rear ____。
- 从一个栈顶指针为top的非空链式栈中删除结点并不需要返回栈顶结点的值和回收结点时，应执行____ top->next = top->next->next ____操作。
- 向一个栈顶指针为top的链式栈中插入一个新结点*p时，应执行___ p->next = top _____和___ top = p _____操作。

  //没有头结点

练习

- 使用没有头结点的循环链表来表示队列，说明如何使用一个指针p操作队列的插入(enqueue)和删除(dequeue)算法，并给出p为何值时队列空
  - 指针p为队列的rear，则rear->next为front //队列要从后面进，所以初始化时p指向尾部
- Enqueue
  - newnode->next= p->next， p->next = newnode，p=newnode，插入结点为第一个节点时需要特殊处理： p= new node, p->next = p
- Dequeue
  - delnode = p->next， p->next = delnode->next , free(delnode) , 删除结点为仅剩结点时需要特殊处理： delnode = p, p =NULL, free(delnode) ;
- 空
  - p==NULL

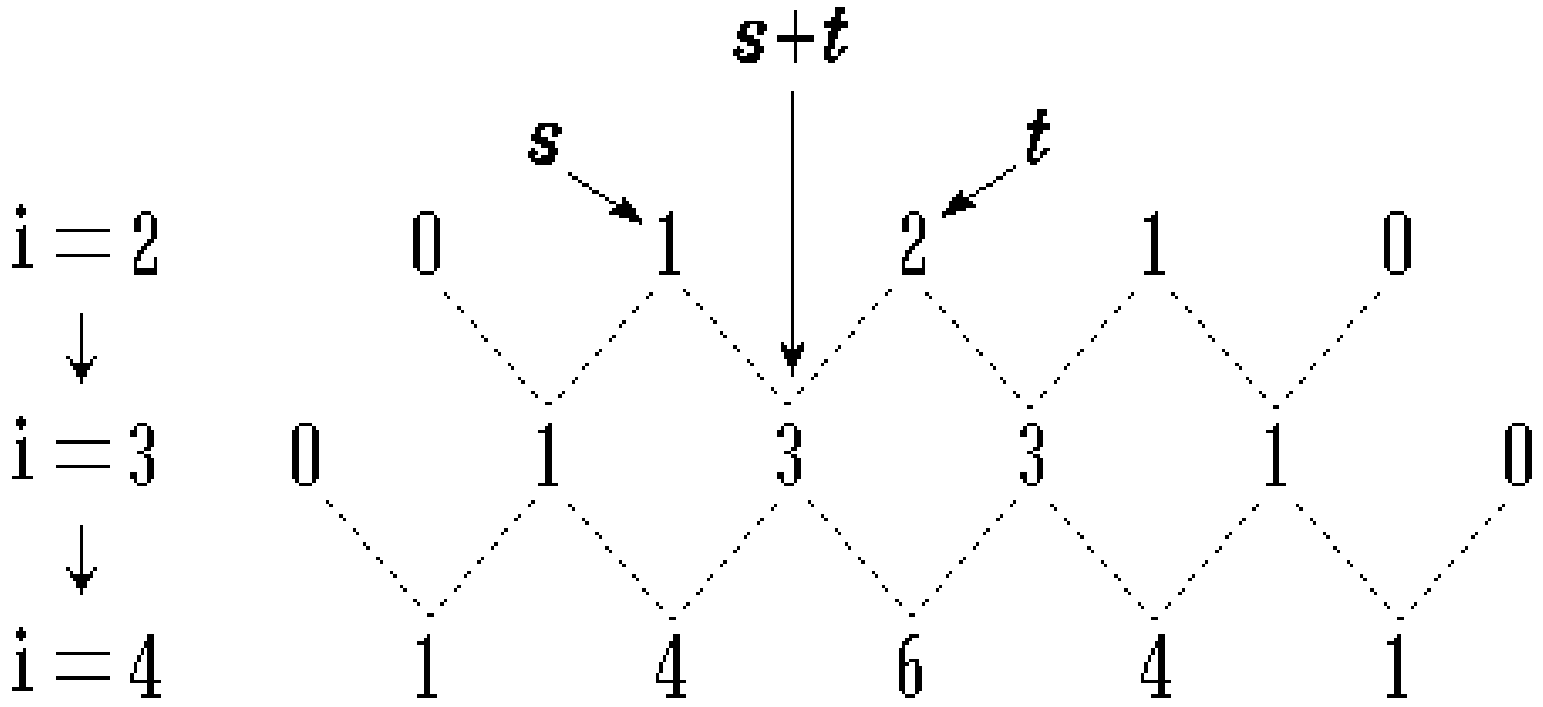


队列应用举例
—打印二项展开式 $(a + b)^i$ 的系数

杨辉三角形 (Pascal's triangle)

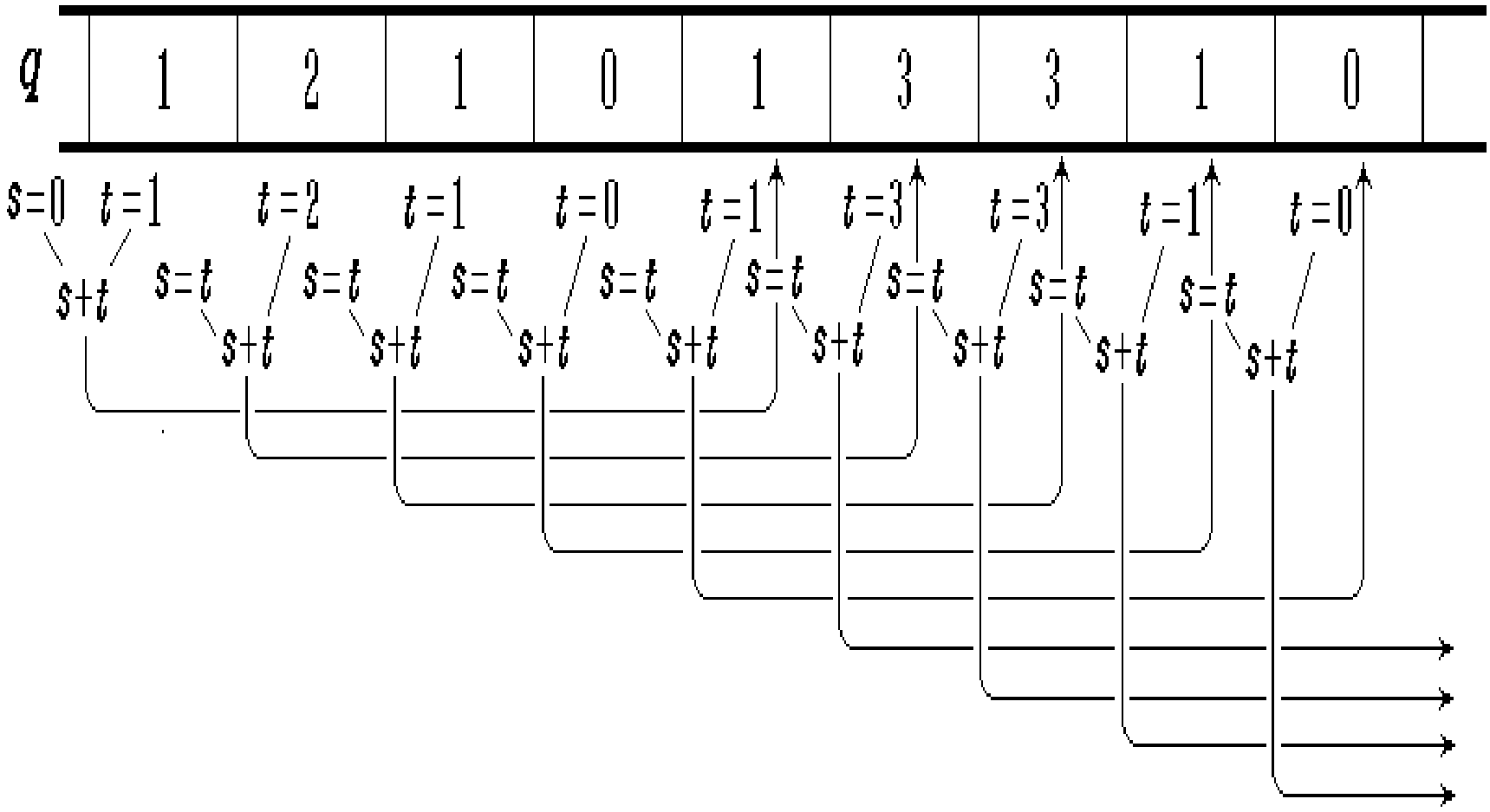
			1	1				$i=1$
		1	2	1				2
	1	3	3	1				3
1	4	6	4	1				4
1	5	10	10	5	1			5
1	6	15	20	15	6	1		6

分析第 i 行元素与第 i+1行元素的关系



目的是从前一行的数据可以计算下一行的数据

从第 i 行数据计算并存放第 i+1 行数据



```
void YANGHUI ( int n ) {
    Queue q;                                  //队列初始化
    q.MakeEmpty ( );
    q.EnQueue (1);  q.EnQueue (1);//预放入第一行的两个系数
    int s = 0;
    //逐行处理，输出第1行参数，计算第2行参数开始
    for ( int i=1; i<=n; i++ ) {
        cout << endl;                         //换行
        q.EnQueue (0);
        for ( int j=1; j<=i+2; j++ ) {  //处理第i行的i+2个系数
            int t = q.DeQueue ( );//读取系数
            q.EnQueue ( s+t );//计算下一行系数，并进队列
            s = t;
            if ( j != i+2 )
            printf("%d  ", s); 打印一个系数，第 i+2个为0
        }
    }
}
```

队列应用-离散事件模拟

日常生活中排队活动的模拟程序是队列的典型应用,用到队列和线性表的数据结构。

设银行有四个窗口，从开门起每个窗口一个时刻只能接待一个客户，人多时客户需排队。当客户进入银行时，如有窗口空闲则直接办理业务，否则会排在人数最少的队伍后面。

编程模拟银行的业务活动并计算一天中客户在银行的平均逗留时间。

思路：

1. **求平均逗留时间要知道每个客户到达和离开银行的两个时刻，所有客户逗留时间总和除以客户数便是平均时间。**
2. **客户到达和离开银行两个时刻发生的事情称为“事件”，整个程序按事件发生的先后顺序进行处理，即事件驱动模拟。**

1.算法中处理的主要对象为“事件”，事件的主要信息是事件类型和发生时刻。

事件有两类：

客户到达事件，发生时刻随客户到来形成；

客户离开事件，发生时刻由客户事务所需时间和等待时间决定。

事件驱动是按事件发生时刻的先后顺序进行，则事件表是有序表，主要操作是插入和删除事件，使用线性表实现。

算法中处理的主要对象为“事件”，事件的主要信息是事件类型和发生时刻。

类型定义如下：

```
//事件类型，有序链表LinkList的数据元素类型
typedef struct {
    int OccurTime; //事件发生时刻
    int NType;       //事件类型，0表示到达事件，1至4表示四个窗口的离开事件
}Event, ElemType;

typedef LinkList EventList //事件链表类型，定义为有序链表
```

2.模拟程序的另一种数据结构是表示客户排队的队列，每个队列的队头是正在办理事务的客户。

银行的四个窗口对应四个队列，队列中客户的信息是其到达的时刻和办理事务所需时间。

```
typedef struct {
   int ArrivalTime; //到达时刻
   int Duration; //办理事务所需时间
}QElemType; //队列的数据元素类型
```

3.队列的队头是正在办理事务的客户，他办完事务离开队列的时刻就是即将发生的客户离开事件的时刻。

即对每个队头客户都存在一个将要驱动的客户离开事件

离开事件的时刻=队头出队时刻+事务办理时间

在任何时刻即将发生的事件只有五种可能：

（1）新客户到达；

（2）1号窗口客户离开；

（3）2号窗口客户离开；

（4）3号窗口客户离开；

（5）4号窗口客户离开。

由以上分析：模拟程序中需要两种数据类型：有序链表和队列。

事件表的数据元素生成过程：

设第一个客户进门的时刻为0，即是模拟程序处理的第一个事件，之后每个客户到达的时刻在前一个客户到达时设定。

在客户到达事件发生时产生两个随机数：

其一为此时刻到达的客户办理事务所需时间durtime;

其二为下一客户将到达的时间间隔intertime

事件表的数据元素生成过程：

假设当前事件发生的时刻为occurtime，则下一个客户到达事件发生的时刻为occurtime+intertime，产生一个新的客户到达事件插入事件表（类型0）

刚到达的客户应插入到当前所含元素最少的队列中；若该队列在插入前为空，则产生一个客户离开事件插入事件表（类型1-4）

客户离开事件在队头元素出队时产生，即下一个个客户开始接受服务时生成，计算该客户在银行逗留的时间，从队列中删除该客户，查看队列是否空,若不空则设定一个新的队头客户离开事件

银行事件驱动模拟程序算法:

```
EventList ev;      //事件表
Event    en;       //事件
LinkQueue        q[4];     //4个客户队列
QElemType        customer;          //客户记录
int TotalTime, CustomerNum;     //累计客户逗留时间，客户数

int cmp (Event a, Event b); //依事件a的发生时刻<或=或>事件b的发
    生时刻分别返回-1或0或1

void OpenForDay(){ //初始化操作
    TotalTime=0; CustomerNum=0; //初始化累计时间和客户数为0
    InitList(ev);   //初始化事件链表为空表
    for (i=0; i<4; ++i) InitQueue(q[i]); //置空队列

    en.OccurTime=0; en.NType=0; //设定第一个客户到达事件
    OrderInsert(ev, en, (* cmp)()); //插入事件表
    } //OpenForDay
```

```
void Bank_Simulation(int CloseTime){
    OpenForDay; //初始化
    while (!EmptyEventList(ev)){
        DelFirst( GetHead(ev),p); en=GetCurElem(p);
        if (en.NType==0)
            CustomerArrived; //处理客户到达事件
        else
            CustomerDeparture; //处理客户离开事件
    }
    //计算并输出平均逗留时间
    printf("The Average Time is %f \n", TotalTime /
    CustomerNum);
} //Bank_Simulation
```

```
void CustomerArrived() { //处理客户到达事件，en.NType=0
    ++CustomerNum;
    Random(durtime, intertime); //生成随机数
    t=en.OccurTime+intertime; //下一客户到达时刻
    if (t<CloseTime)    //银行尚未关门，插入事件表
        OrderInsert(ev,(t,0),(* cmp)());

    i=Minimun(q);       //求长度最短队列
    EnQueue(q[i], (en.OccurTime, durtime));
    if (QueueLength(q[i]) == 1)
        OrderInsert (ev, (en.OccurTime+durtime, i), (*
    cmp)()); //设定第i队列的一个离开事件并插入事件表
} //CustomerArrived
```

生成下一个客户的到达事件插入事件表，当前客户进最短队列，如果队列只剩一人则生成其离开事件加入事件表

```
Void CustomerDeparture() {
   //处理客户离开事件，  en.NType>0
   i=en.NType;
   //删除第i队列的排头客户
   DelQueue(q[i], customer);
   TotalTime+=en.OccurTime –
   customer.ArrivalTime; //累计客户逗留时间
   //设定第i队列的下一个离开事件并插入事件表
   if (!QueueEmpty(q[i])){
        GetHead(q[i], customer);
        OrderInsert (ev,
   (en.OccurTime+customer.Duration, i), (*
   cmp());
   } //if
} //CustomerDeparture
```

删除队头客户并计算时间，生成下一个队头客户的离开事件加入事件表

递 归

定义 若一个对象部分地包含它自己，或用它自己给自己定义，则称这个**对象是递归的**；
若一个过程直接地或间接地调用自己，则称这个**过程是递归的**过程。

三种递归情况

- 定义是递归的
- 数据结构是递归的
- 问题的解法是递归的

定义是递归的

例1.阶乘函数

$$n! = \begin{cases} 1, & \text{当 } n = 0 \text{ 时} \\ n * (n-1)!, & \text{当 } n \geq 1 \text{ 时} \end{cases}$$

求解阶乘函数的递归算法

```
long Factorial ( long n ) {
    if ( n == 0 ) return 1;
    else return n * Factorial (n−1);
}
```

递归要求需要有出口

求解阶乘 n! 的过程


主程序 main : fact(4)
参数 4　计算 4*fact(3)　返回 24
参数 3　计算 3*fact(2)　返回 6
参数 2　计算 2*fact(1)　返回 2
参数 1　计算 1*fact(0)　返回 1
参数 0　直接定值 = 1　返回 1
递归调用
参数传递
结果返回
回归求值

例2.计算斐波那契数列函数Fib(n)的定义

$$\mathrm{Fib}(n)=\begin{cases} n, & n=0,1 \\ \mathrm{Fib}(n-1)+\mathrm{Fib}(n-2), & n>1 \end{cases}$$

如 $F_0=0, F_1=1, F_2=1, F_3=2, F_4=3, F_5=5$

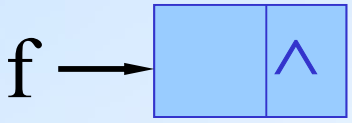
递归算法

```
long Fib ( long n ) {
    if ( n <= 1 ) return n;
    else return Fib (n−1) + Fib (n−2);
}
```

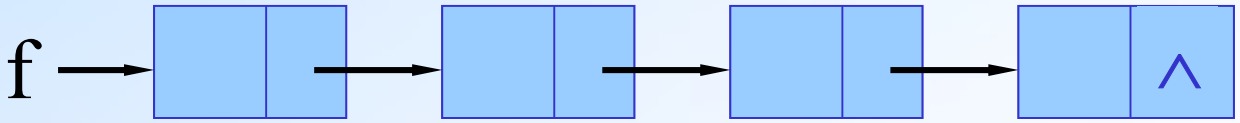
数据结构是递归的

例如单链表结构

只有一个结点的单链表



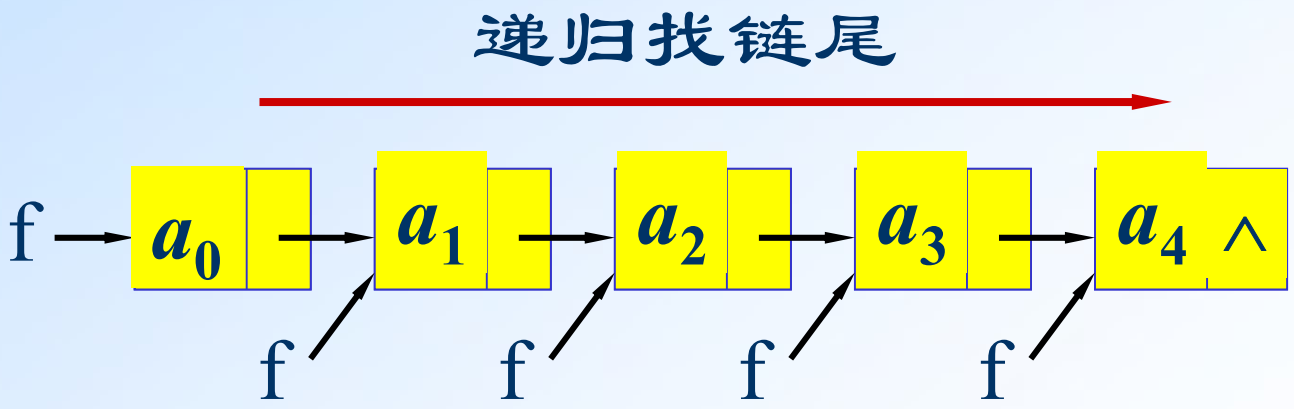
有若干结点的单链表



通过->link来递归

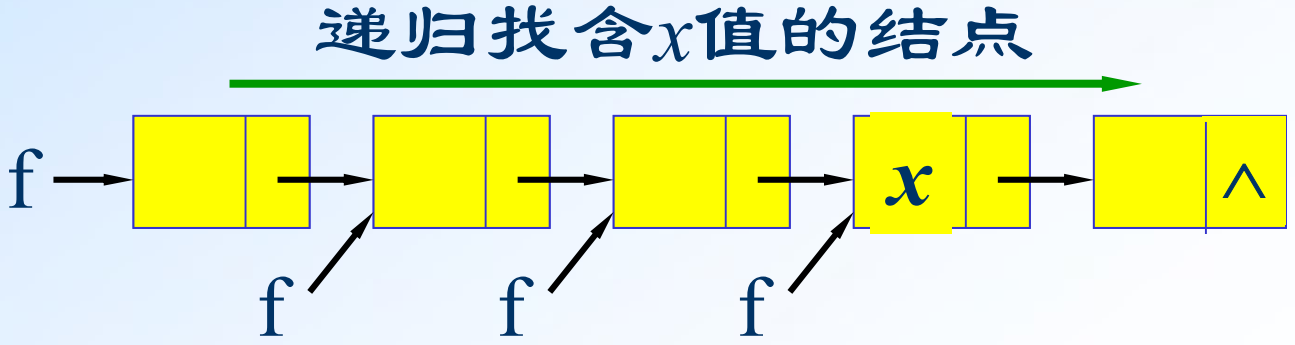
例1. 搜索链表最后一个结点并打印其数值

```
void Search ( ListNode *f ) {
    if ( f ->link == NULL )
            printf("%d \n", f->data);
    else Search ( f ->link );
}
```



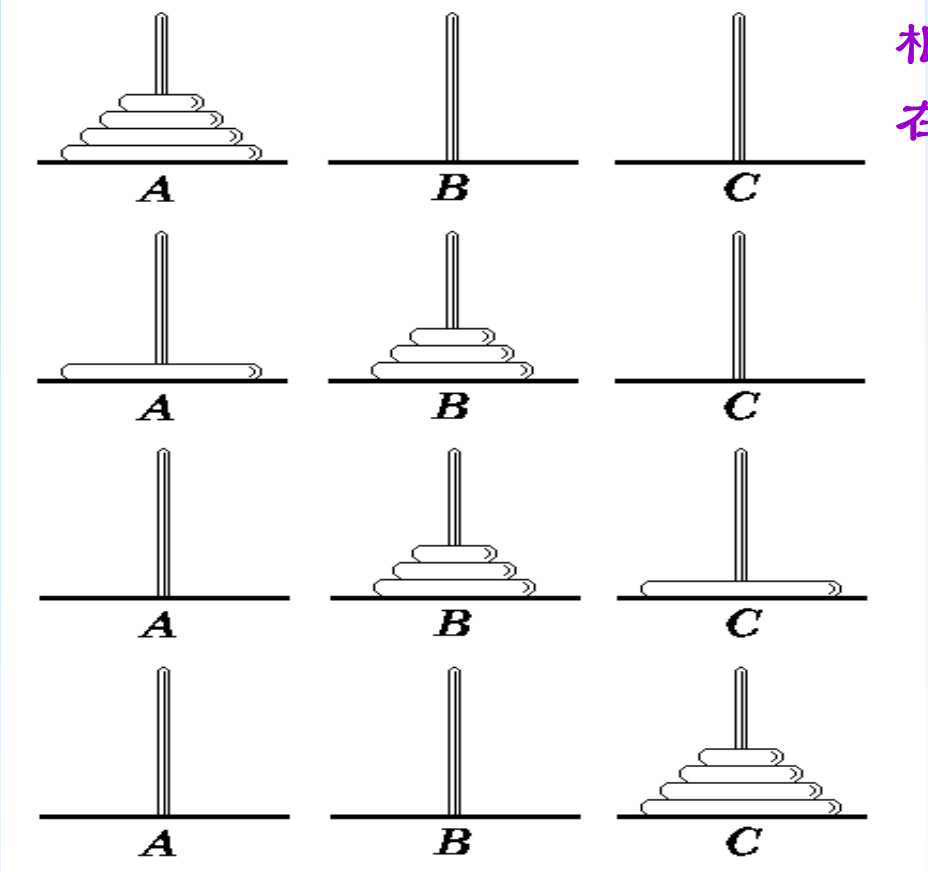
例2.在链表中寻找等于给定值的结点,并打印其数值

```
void Search ( ListNode *f, Type& x ) {
    if ( f != NULL )
        if ( f -> data == x )
                printf("%d \n", f->data);
        else Search ( f -> link, x );
}
```



问题的解法是递归的

汉诺塔(Tower of Hanoi)问题



A移到C，一次只移动一片，不管在哪根针上，小片必须在大片上面。

问题的解法是递归的

汉诺塔(Tower of Hanoi)问题的解法：

如果 n = 1，则将这一个盘子直接从 A 柱移到 C 柱上。否则，执行以下三步：

① 用 C 柱做过渡，将 A 柱上的 (n-1) 个盘子移到 B 柱上：

② 将 A 柱上最后一个盘子直接移到 C 柱上；

③ 用 A 柱做过渡，将 B 柱上的 (n-1) 个盘子移到 C 柱上。

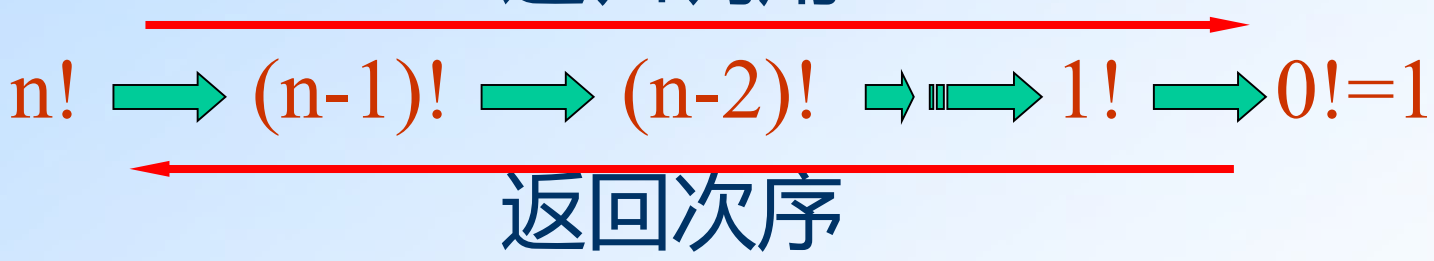
C语言描述

```
Void hanoi(int n, char x, char y,char z)
{//将塔座x上按直径从小到大且自上而下编号为1到n的n个圆盘按规则
//搬到塔座z上，y可以用作辅助塔座
        {
            if(n==1) move(x,1,z); //将编号为1的圆盘从x移到z
            else{
                hanoi(n-1,x,z,y);//将x上编号为1到n-1的圆盘移到y,z作辅助塔
                move(x,n,z); //将编号为n的圆盘从x移到z
                hanoi(n-1,y,x,z);//将y上编号为1到n-1的圆盘移到z,x作辅助塔
            }
        }
}
```

递归过程

递归过程在实现时，需要自己调用自己。

层层向下递归，退出时的次序正好相反：



主程序第一次调用递归过程为外部调用；

递归过程每次递归调用自己为内部调用。

递归过程使用的数据结构是什么？

递归工作栈

每一次递归调用时，需要为过程中使用的参数、局部变量等另外分配存储空间。

每层递归调用需分配的空间形成递归工作记录，按后进先出的栈组织。



```
                long Factorial ( long n ) {
①                   int temp;
②                   if ( n == 0 ) return 1;
③                   else return n * Factorial (n-1);
   RetLoc2 ──────────────────────┘   内部调用

④                   }
void main ( ) {
①     int n;
②     n = Factorial (4);
   RetLoc1 ──────┘   外部调用
}
```

计算Fact时活动记录的内容



递归过程改为非递归过程

√ 递归过程简洁、易编、易懂

× 递归过程效率低，重复计算多

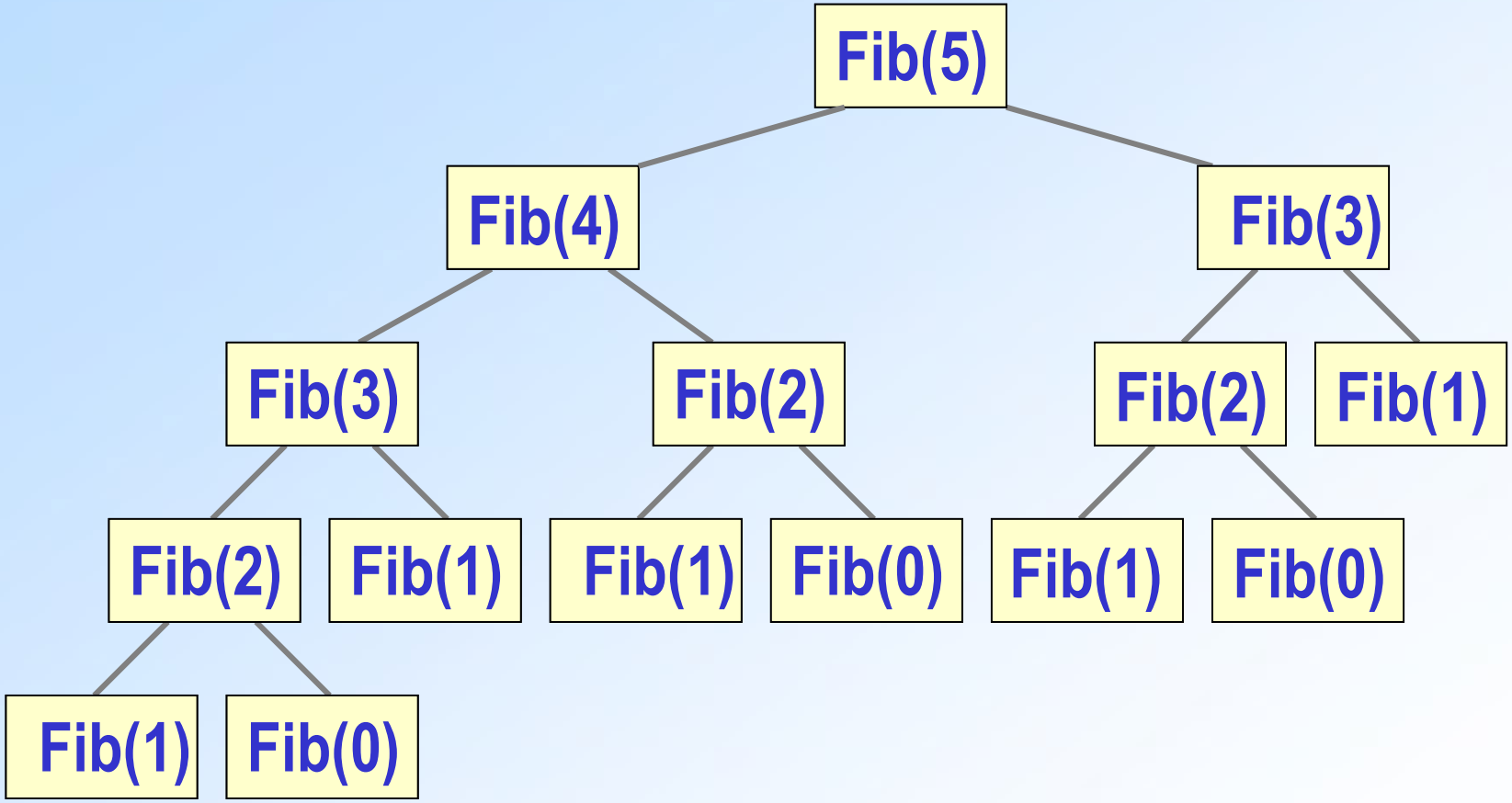
改为非递归过程的目的是提高效率

- 单向递归和尾递归可直接用迭代（循环）实现其非递归过程
- 其他情形借助栈实现非递归过程

尾递归：递归调用语句只有一个，而且是处于算法的最后。如 **求阶乘算法**。

单向递归：程序中的递归语句，在本程序操作执行前，都已经完成。指递归函数中虽然有一处以上的递归调用语句，但各次递归调用语句的参数只和主调用函数有关，相互之间参数无关，并且这些递归调用语句，也与尾递归一样处于算法的最后。如 **Fibonacci数列**。

递归过程效率低，重复计算多



斐波那契数列的递归调用树

尾递归用迭代法实现

25	36	72	18	99	49	54	63

```
void recfunc ( int A[ ],  int n ) {
//从n到0输出数组中各项的值
   if ( n >= 0 ) {
         printf("%d  ", A[n]);
         n--;
      recfunc ( A, n );
   }
}
```

```
void sterfunc ( int A[ ], int n ) {
//消除了尾递归的非递归函数
   while ( n >= 0 ) {
         printf("value  %d \n", A[n]);
      n--;
   }
}
```

计算斐波那契数列的函数Fib(n)的定义

$$\mathrm{Fib}(n)=\begin{cases} n, & n=0,1 \\ \mathrm{Fib}(n-1)+\mathrm{Fib}(n-2), & n>1 \end{cases}$$

如 $F_0=0, F_1=1, F_2=1, F_3=2, F_4=3, F_5=5$

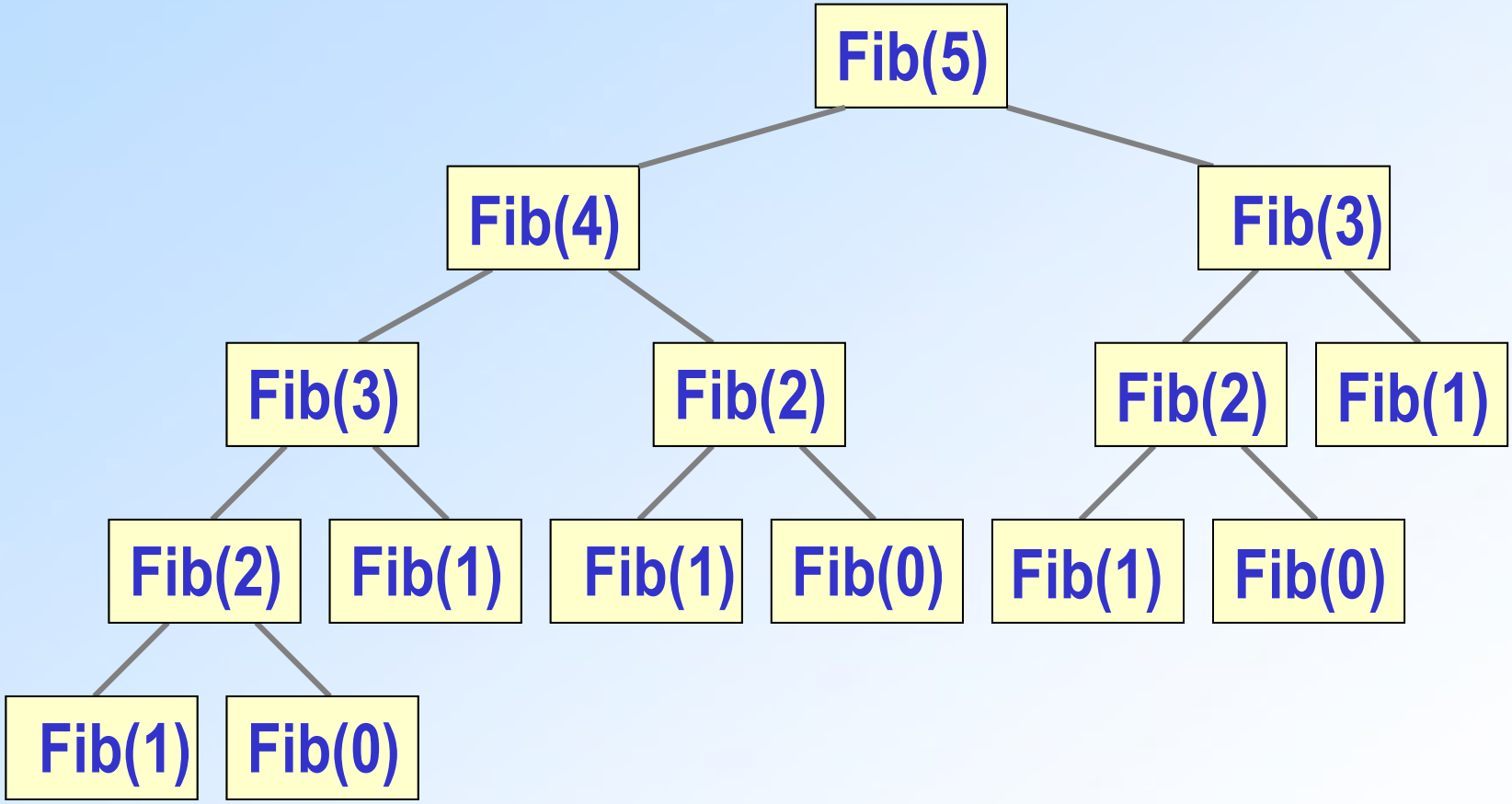
求解斐波那契数列的递归算法

```
long Fib ( long n ) {
    if ( n <= 1 ) return n;
    else return Fib (n−1) + Fib (n−2);
}
```

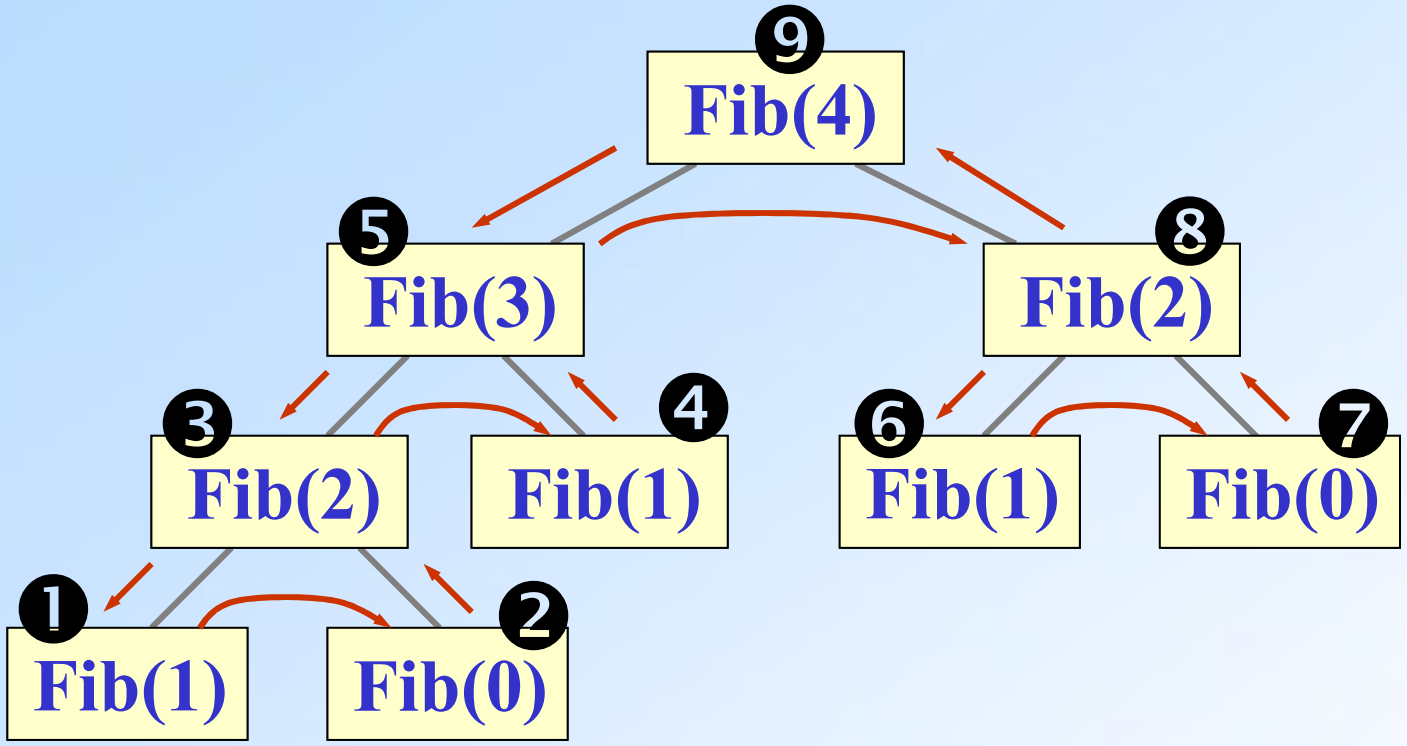
单向递归用迭代法实现

```
long FibIter ( long n ) {
    if ( n <= 1 ) return n;
    long twoback = 0,  oneback = 1,  Current;
    for ( int i = 2; i <= n; i++ ) {
      Current = twoback + oneback;
      twoback = oneback;
      oneback = Current;
    }
    return Current;
}
```

❖使用栈进行递归转换　斐波那契数列的递归调用树



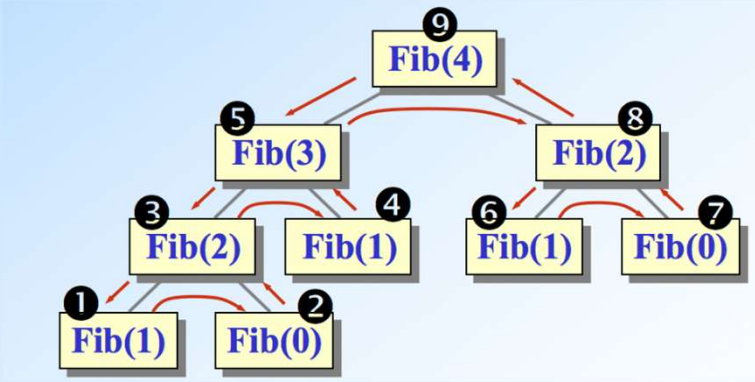
可以利用栈，将Fib()压入栈。先计算Fib(1),从Fib(4)一直向左走下去。为了回退，需要用栈存储回退的路径，以便计算。为了区分从左退回还是从右退回，需要在结点增加一个标志信息tag。N大于1的左支路压栈，按压栈顺序计算右支路并求和



栈结点

n	tag

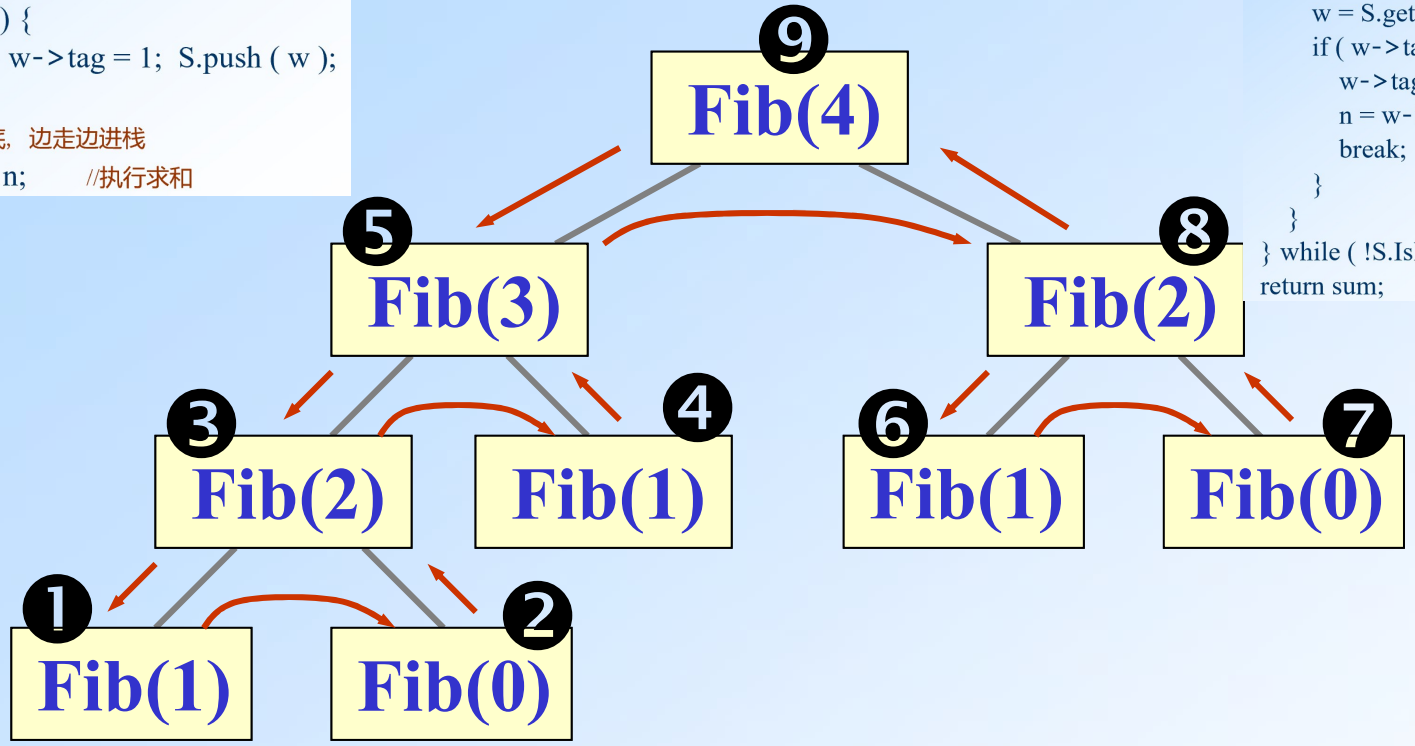
tag = 1, 向左递归； **tag = 2**, 向右递归

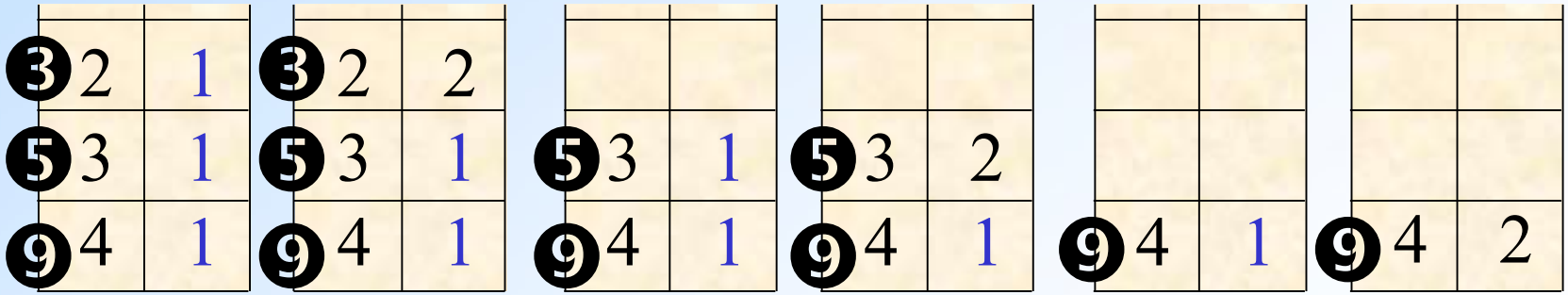



```
long Fibnacci ( long n ) {
    Stack S;   Node *w;  long sum = 0;
    //反复执行直到所有终端结点数据累加完
    do {
       while ( n > 1 ) {
          w->n = n;  w->tag = 1;  S.push ( w );
          n--;
       } //向左递归到底，边走边进栈
       sum = sum + n;          //执行求和
```

```
        while ( !S.IsEmpty( ) ) {
            w = S.getTop( );  S.Pop( );
            if ( w->tag == 1 ) {      //改为向右递归
                w->tag = 2;  S.push ( w );
                n = w->n – 2;      //F(n)右侧为F(n-2)
                break;
            }
        }
    } while ( !S.IsEmpty( ) );
    return sum;
}
```

```
do {
  while ( n > 1 ) {
    w->n = n;  w->tag = 1;  S.push ( w );
    n--;
  } //向左递归到底，边走边进栈
  sum = sum + n;        //执行求和
  while ( !S.IsEmpty( ) ) {
    w = S.getTop( );  S.Pop( );
    if ( w->tag == 1 ) {     //改为向右递归
      w->tag = 2;  S.push ( w );
      n = w->n – 2;      //F(n)右侧为F(n-2)
      break;
    }
  }
} while ( !S.IsEmpty( ) );
return sum;
```





1 n=1 sum=0+1

2 n=2-2

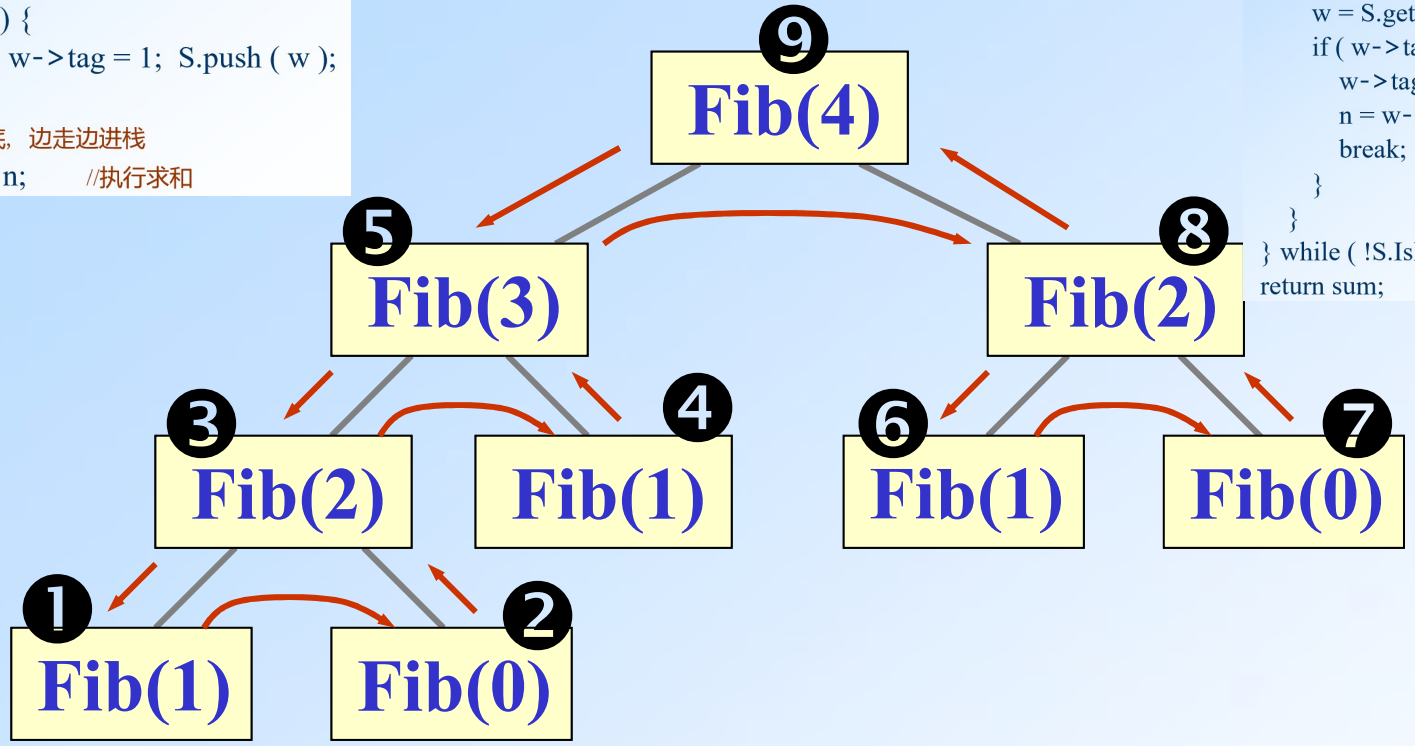
2 n=0 sum=1+0

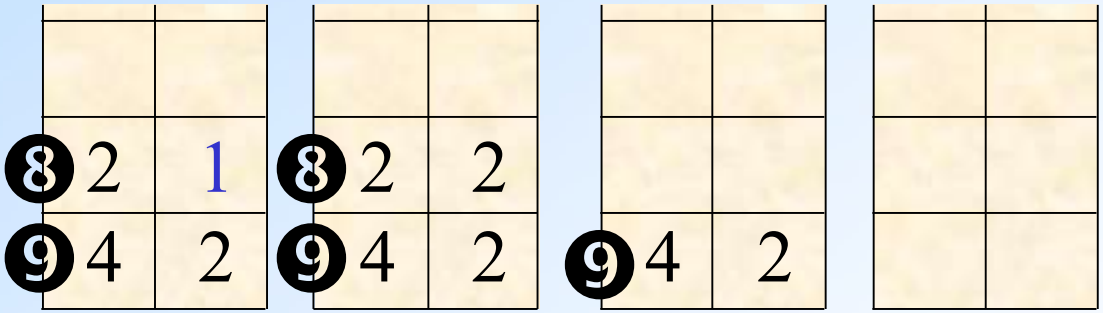
4 n=3-2

4 n=1 sum=1+1

8 n=4-2

```
do {
  while ( n > 1 ) {
    w->n = n;  w->tag = 1;  S.push ( w );
    n--;
  } //向左递归到底，边走边进栈
  sum = sum + n;        //执行求和
  while ( !S.IsEmpty( ) ) {
    w = S.getTop( );  S.Pop( );
    if ( w->tag == 1 ) {     //改为向右递归
      w->tag = 2;  S.push ( w );
      n = w->n – 2;      //F(n)右侧为F(n-2)
      break;
    }
  }
} while ( !S.IsEmpty( ) );
return sum;
```





6 n=1 sum=2+1

7 n=2-2

7 n=0 sum=3+0

1+0+1+1+0

Sum把所有的叶结点相加

按照左右的顺序访问所有叶结点

❖递归与回溯

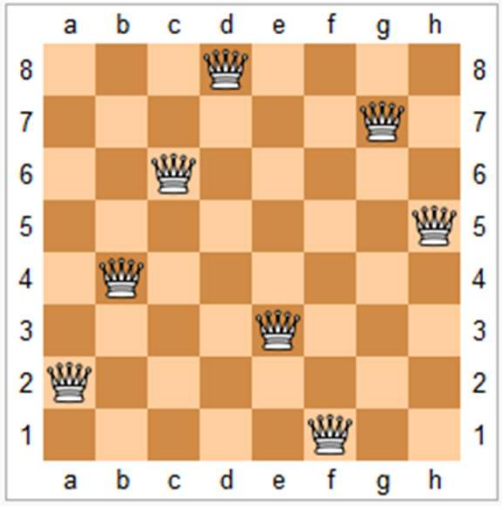
递归只是返回值，但是回溯还需要有状态的重置

n皇后问题

在 n 行 n 列的国际象棋棋盘上，若两个皇后位于同一行、同一列、同一对角线上，则称为它们为互相攻击。n皇后问题是指找到这 n 个皇后的互不攻击的布局。

如何将 n 个皇后放置在 n × n 的棋盘上，并且使皇后彼此之间不能相互攻击。

每个皇后必须各占一行。

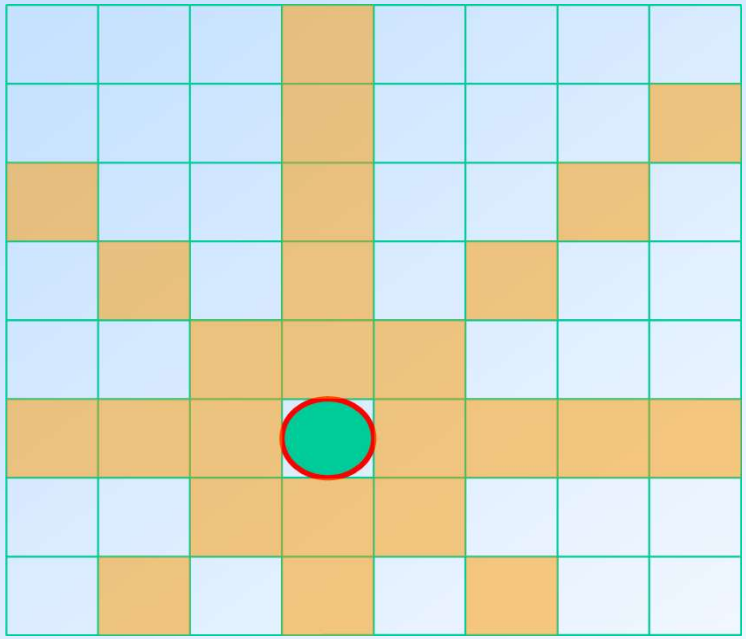


One solution to the eight queens puzzle

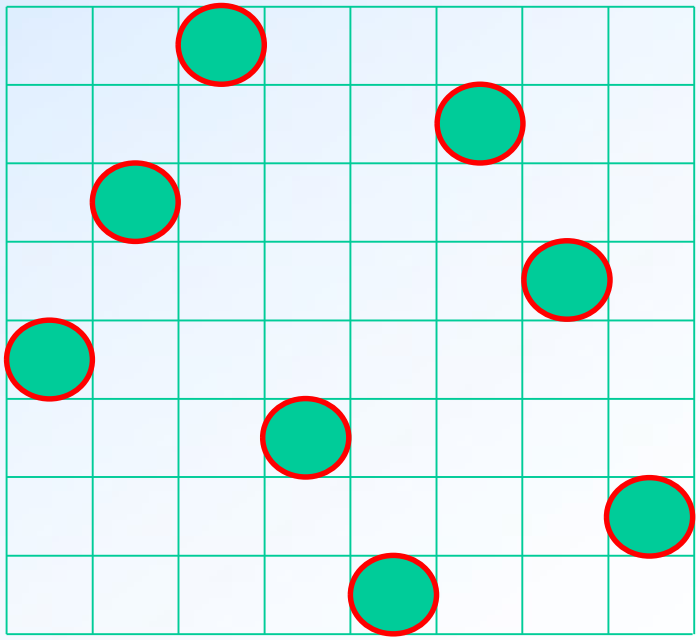
❖递归与回溯

n皇后问题

在nxn棋盘上放置n个（n>4）皇后，如何保证她们互相不攻击



皇后的控制范围



8皇后问题的一个可行解（解不唯一）

解题思路

安放第 i 行皇后时，需要在列的方向从 0 到 n-1 试探 (j = 0, …, n-1)

在第 j 列安放一个皇后：

- 如果在列、主对角线、次对角线方向有其它皇后，则出现攻击，撤消在第 j 列安放的皇后
- 如果没有出现攻击，在第 j 列安放的皇后不动，递归安放第 i+1行皇后

设置 4 个数组

- col [n]： col[i] 标识第 i 列是否安放了皇后
- md[2n-1] : md[k] 标识第 k 条主对角线是否安放了皇后
- sd[2n-1] : sd[k] 标识第 k 条次对角线是否安放了皇后
- q[n] : q[i] 记录第 i 行皇后在第几列

```
void Queen( int i ) {  //安放第i行皇后
    for ( int j = 0;  j < n;  j++ ) {
        if ( 第 i 行第 j 列没有攻击 ) {
            在第 i 行第 j 列安放皇后;
             if ( i == n-1 ) 输出一个布局;
             else Queen ( i+1 );
            撤消第 i 行第 j 列的皇后;
        }
    }
}
```

```
void Queen( int i ) {
    for ( int j = 0;  j < n;  j++ ) {
        if ( !col[j] && !md[n+i-j-1] && !sd[i+j] )
        {       /*第 i 行第 j 列没有攻击 */
                col[j] = md[n+i-j-1] = sd[i+j] = 1;
                q[i] = j; /*在第 i 行第 j 列安放皇后*/
                if ( i == n-1 ) {    /*输出一个布局*/
                   for ( int k = 0; k < n; k++ ) {
                          printf(" %d , \n", q[k]);
                          //return; }
                }                          有return输出一种可能，没
                else  Queen ( i+1 ); return输出所有可能

                col[j] = md[n+i-j-1] = sd[i+j] = 0;
                q[i] = 0;  /*撤消第 i 行第 j 列的皇后*/
            }
        }
}
```

骑士问题

问题：在N行N列的棋盘上，一位骑士按象棋中“马走日”的走法从初始坐标位置（SX, SY）出发，要求遍历（巡游）棋盘中每一个位置一次。请输出骑士巡游的位置顺序，或输出无解。

	4		3	
5				2
		(X_0,Y_0)		
6				1
	7		0	

1. 将这八个可能的移动方向使用数组来存放。

```
Typedef struct {
        int vert;
        int horiz;
}offsets;
offsets move[8];
```

若假设目前的位置在maze[x][y]，而下一步的位置在maze[next_x][next_y]，则可利用direct变量知道next_x和next_y两个值：

```
next_x=x+move[direct].vert
next_y=y+move[direct].hor
        iz
```

方向索引值 (direct)	move[direct].vert	move[direct].horiz
0	2	1
1	1	2
2	-1	2
3	-2	1
4	-2	-1
5	-1	-2
6	1	-2
7	2	-1

2.在移动之前必须判断下一步有没有走出了边界：

```
int in_range( int x, int y) {
    return( x>=0 && x<MAX_ROW && y>=0 &&
    y<MAX_COL );
}
```

当骑士走到(x,y)时，**下一个移动的位置应该是这八个位置中的其中一个**，我们便从0这个方向开始走，如果新位置未走过的话，就可以移到新位置上，并记录下所走的步数，如果新位置已被走过，则继续找下一个方向，即将direct变量加1。

由于一个N*N的棋盘，骑士必须**走N^2 - 1步**，才能将棋盘上的每一点都走过，即棋盘的大小减去一开始的起始位置那一点。因此若我们设起点为第一步，则全部走完时最后一步的步数会刚好是棋盘的大小N*N，若所走的步数等于N * N的话，代表骑士已经全部走完。

```
void tour (int count, int x, int y ){
    int direct, next_x, next_y;
    direct=0;
    while (direct<8)
    {       next_x=x+move[direct].vert;
            next_y=y+move[direct].horiz;
            if (in_range (next_x, next_y)){
                    if (maze[next_x][next_y]==0)
                    {           maze[next_x][next_y]=count;
                                if (count==N*N)
                                        print_ans();
                                else
                                        tour (count+1,next_x,next_y);
                                maze[next_x][next_y]=0;  //回溯 重置
                    }
            }
            direct++;
    }
}
```

```
void print_ans(){
    int i,j;
    for(i=0;i<N;i++,printf("\n"))
        for(j=0;j<N;j++)
            printf("%6d",maze[i][j];
        printf("\n\n");
}
```

初始count = 2，因为第一格为第一步1

练习

用递归法将10进制数转换为8进制数.

练习

用递归法将10进制数转换为8进制数.

```
void conversion (int n){
    int count=0;
    if(n==0){
        return ;
    }
    else{
        conversion(n/8);
        printf("%d",n%8);
}
```

迷宫求解

- 递归方法

迷宫求解

我们用矩阵表示地图：其中1表示路，0表示没有路，2表示终点，起始地点为（1,0）。2代表出口，#代表结点已经走过

```
void find_path_all(int x, int y)
{
    if(check_pos_valid(x,y))
    {
        if(2 == gPath[x][y]){
            gValue[x][y] = '#';
            print_path();
            gValue[x][y] = 0;
            return ;
        }

        gValue[x][y] = '#';
        find_path(x, y-1);
        find_path(x-1, y);
        find_path(x, y+1);
        find_path(x+1, y);
        gValue[x][y] = 0;
    }
}
```

遍历所有的通路

```
int find_path(int x, int y)
{
  if(check_pos_valid(x,y))
  {
    if(2 == gPath[x][y]){
      gValue[x][y] = '#';
      print_path();
      return 1;
    }

    gValue[x][y] = '#';
    if(find_path(x, y-1))
      return 1;

    if(find_path(x-1, y))
      return 1;

    if(find_path(x, y+1))
      return 1;

    if(find_path(x+1, y))
      return 1;

    gValue[x][y] = 0;  //回溯
    return 0;
  }

  return 0;
}
```

寻找一条通路

https://developer.aliyun.com/article/263060

练习

McCathy函数定义如下：

$$M(x)=\begin{cases} x-10 & x>100 \\ M(M(x+11)) & x\le 100 \end{cases}$$

用递归和非递归方法实现

麦卡锡函数：当输入一个大于 100 的数字 x 时返回 x-10，当 x<100 的时候递归的对 x 进行运算，但是结果总是 91。

练习

McCathy函数定义如下：

$$M(x)=\begin{cases} x-10 & x>100 \\ M(M(x+11)) & x\le 100 \end{cases}$$

用递归和非递归方法实现

```
int McCarthy91(const int& value){
    if(value>100)
        return value-10;
    else
        return McCarthy91(McCarthy91(value+11));
}
```

练习

McCathy函数

$$M(x)=\begin{cases} x- \\ M(M(x \end{cases}$$

用递归和非递

```
void McCarthy91_R(const int& value){
    myStack ms;
    int i = value;
    ms.push('0');
    while(!ms.isEmpty()){
        ms.pop();
        if(i>100)
            i -= 10;
        else{
            ms.push('0');
            ms.push('0');
            i += 11;
        }
    }
    std::cout<<i<<std::endl;
}
```

结束

栈的抽象数据类型

栈的抽象数据类型定义为：

ADT Stack {

数据对象： $D=\{a_i \mid a_i \in ElemSet,\ i=1,2,...,n,\ n\geqslant 0\}$

数据关系： $R1=\{<a_{i-1},\ a_i> \mid a_{i-1},\ a_i \in D,\ i=2,...,n\}$，**约定**$a_n$**端为栈顶，**$a_1$**端为栈底。**

基本操作：

……

} ADT Stack

栈的抽象数据类型（C++）

```
template <class Type> class Stack {
public:
    Stack ( int sz = 10 );          //构造函数
    void Push ( Type& item);        //进栈
    Type Pop ( );                   //出栈
    Type GetTop ( );                //取栈顶元素
    void MakeEmpty ( );             //置空栈
    int IsEmpty ( ) const;          //判栈空否
    int IsFull ( ) const;           //判栈满否
}
```

栈的数组表示 — 顺序栈（C++）

assert的作用是计算表达式 expression，其值为假（即为0），则先向stderr打印一条出错信息，然后通过调用 abort 来终止程序运行。

```
#include <assert.h>
template <class Type> class Stack {
private:
    int top;                        //栈顶指针
    Type *elements;                 //栈元素数组
    int maxSize;                    //栈最大容量
public:
    Stack ( int sz = 10 );          //构造函数
    ~Stack ( ) { delete [ ] elements; }
    void Push ( Type & item );      //进栈
```

```
    Type Pop ( );                          //出栈
    Type GetTop ( );                       //取栈顶
    void MakeEmpty ( ) { top = -1; }   //置空栈
    int IsEmpty ( ) const { return top == -1; }
    int IsFull ( ) const
        { return top == maxSize-1; }
}

template <class Type> Stack<Type> ::
Stack ( int s ) : top (-1), maxSize (s) {
    elements = new Type[maxSize];
    assert ( elements != NULL );        //断言
}
```

```
template <class Type> void Stack<Type> ::
Push ( Type & item ) {
    assert ( !IsFull ( ) );          //先决条件断言
    elements[++top] = item;          //加入新元素
}

template <class Type> Type stack<Type> ::
Pop ( ) {
    assert ( !IsEmpty ( ) );         //先决条件断言
    return elements[top--] ;         //退出栈顶元素
}
```

```
template <class Type> Type stack<Type>::
GetTop ( ) {
    assert ( !IsEmpty ( ) );        //先决条件断言
    return elements[top];           //取出栈顶元素
}
```

栈顶位置的规定并不是一成不变的

课件的内容理解思想，不要死记硬背

链式栈 (LinkedStack)类的定义（C++）

```
template <class Type> class Stack;

template <class Type> class StackNode {
friend class Stack<Type>;
private:
    Type data;                          //结点数据
    StackNode<Type> *link;              //结点链指针
public:
    StackNode ( Type d = 0, StackNode<Type>
        *l = NULL ) : data ( d ), link ( l ) { }
};
```

```
template <class Type> class Stack {
private:
    StackNode<Type> *top;        //栈顶指针
public:
    Stack ( ) : top ( NULL ) { }
    ~Stack ( );
    void Push ( const Type & item);   //进栈
    int Pop ( );                      //退栈
    Type *GetTop ( );         //读取栈顶元素
    void MakeEmpty ( );   //实现与~Stack( )同
    int IsEmpty ( ) { return top == NULL; }
}
```

```
template <class Type> Stack<Type> ::
~Stack ( ) {
    StackNode<Type> *p;
    while ( top != NULL )        //逐结点回收
        { p = top;  top = top->link;  delete p; }
}

template <class Type> void Stack<Type> ::
Push ( const Type &item ) {
   top = new StackNode<Type> ( item, top );
   //新结点链入*top之前，并成为新栈顶
}
```

```
template <class Type> int Stack<Type> ::
Pop ( ) {
    if ( IsEmpty ( ) ) return 0;
    StackNode<Type> *p = top;
    top = top->link;                    //修改栈顶指针
    delete p;   return 1;               //释放
}

template <class Type> Type Stack<Type>::
GetTop ( ) {
    assert ( !IsEmpty ( ) );
    return top->data;
}
```



优先队列的类定义(C++)

```
#include <assert.h>
#include <iostream.h>
#include <stdlib.h>
const int maxPQSize = 50;     //缺省元素个数

template <class Type> class PQueue {
public:
    PQueue ( );
    ~PQueue ( ) { delete [ ] pqelements; }
    void PQInsert ( const Type & item );
    Type PQRemove ( );
```

```
    void makeEmpty ( ) { count = -1; }
    int IsEmpty ( ) const
          { return count == -1; }
    int IsFull ( ) const
          { return count == maxPQSize; }
    / /优先级队列元素个数
    int Length ( ) const { return count; }
private:
    Type *pqelements; //存放数组（优先级队列数组）
    int count;                    //队列元素计数
}
```

优先级队列部分成员函数的实现

```
template <class Type>
PQueue<Type>::PQueue ( ) : count (−1) {
    pqelements = new Type[maxPQSize];
    assert ( pqelements != 0 );   //分配断言
}       //创建优先级队列

template <class Type> void PQueue<Type> ::
PQInsert ( const Type & item ) {
    assert ( !IsFull ( ) );              //判队满断言
    count ++;    pqelements[count] = item;
}       //插入元素到队尾
```

```
template <class Type>
Type PQueue<Type>::PQRemove ( ) {
   assert ( !IsEmpty ( ) );            //判队空断言
   Type min = pqelements[0];
   int minindex = 0;
   for ( int i=1; i<count; i++ )  //寻找最小元素
     if ( pqelements[i] < min )   //存于min
          { min = pqelements[i];   minindex = i; }
   pqelements[minindex] = pqelements[count-1];
   //用最后一个元素填补要取走的最小值元素
   count--  ;//优先级队列中元素个数减一
   return min;
}       //从队中删除最小值（优先级最高）元素
```

队中没有顺序要求，因此可以用最后一个元素填补要取走的最小值元素

循环队列的类定义(C++)

```
#include <assert.h>
template <class Type> class Queue {
private:
    int rear, front;                    //队尾，队头指针
    Type *elements;                     //队列元素数组
    int maxSize;                        //最大元素个数
public:
    Queue ( );
    ~Queue ( ) { delete [ ] elements; }
    void EnQueue ( const Type & item); //进队
```

```
    int DeQueue ( );            //退队
    Type *GetFront ( );         //取队头元素
    void MakeEmpty ( ) { front = rear = 0; }
    int IsEmpty ( ) const
      { return front == rear; }
    int IsFull ( ) const
      { return (rear+1) % maxSize == front; }
    int Length ( ) const
      { return (rear−front+maxSize) % maxSize;}
}
```

```
template <class Type>
Queue<Type>:: Queue ( int sz ) :
front (0), rear (0), maxSize (sz) {
    elements = new Type[maxSize];
    assert ( elements != NULL );
}

template <class Type> void Queue<Type>::
EnQueue ( const Type & item ) {
     assert ( !IsFull ( ) );
     elements[rear] = item;
     rear = (rear+1) % MaxSize;
}
```

```
template <class Type>
int Queue<Type> :: DeQueue ( ) {
    if ( IsEmpty ( ) ) return 0;
    front = ( front+1) % MaxSize;
    return 1;
}

template <class Type>
Type Queue<Type> :: GetFront ( ) {
    assert ( !IsEmpty ( ) );
    return elements[front];
}
```

链式队列类的定义(C++)

```
template <class Type> class Queue;

template <class Type> class QueueNode {
friend class Queue<Type>;
private:
    Type data;                          //队列结点数据
    QueueNode<Type> *link;  //结点链指针
public:
    QueueNode ( Type d=0, QueueNode<Type>
      *l = NULL ) : data (d), link (l) { }
};
```

```
template <class Type> class Queue {
private:
    QueueNode<Type> *front, *rear;
                        //队头、队尾指针指针
public:
    Queue ( ) : rear ( NULL ), front ( NULL ) { }
    ~Queue ( );
    void EnQueue ( const Type & item );
    int DeQueue ( );
    Type *GetFront ( );
    void MakeEmpty ( );  //实现与~Queue( )同
```

```
    int IsEmpty ( ) const
                { return front == NULL; }
};

template <class Type>
Queue<Type> :: ~Queue ( ) {
//队列的析构函数
   QueueNode<Type> *p;
   while ( front != NULL ) {   //逐个结点释放
      p = front;  front = front->link;  delete p;
   }
}
```

```
template <class Type>
void Queue<Type> ::
EnQueue ( const Type& item ) {
//将新元素item插入到队列的队尾
    if ( front == NULL )    //创建第一个结点
        front = rear = new QueueNode
                <Type> ( item, NULL );
    else                                //队列不空, 插入
        rear = rear->link = new QueueNode
                <Type> ( item, NULL );
}
```

```
template <class Type>
int Queue<Type> :: DeQueue ( ) {
    if ( IsEmpty ( ) ) return 0;      //判队空
    QueueNode<Type> *p = front;
    front = front->link;   delete p;
    return 1;
}

template <class Type>
Type *Queue<Type> :: GetFront ( ) {
    if ( IsEmpty( ) ) return NULL;
    return & front->data;
}
```

C++描述

```
#include <iostream.h>
#include "strclass.h”
void Hanoi (int n, String A, String B, String C) {
 //解决汉诺塔问题的算法
   if ( n == 1 )
        cout << " move " << A << " to  "  << C << endl;
   else {
        Hanoi ( n−1, A, C, B );
        cout << " move " << A << " to " << C << endl;
        Hanoi ( n−1, B, A, C );
   }
 }
```

